

Оценить безопасность сторонней библиотеки

Сотрудник «Тестирования безопасности» на этапе «Оценка безопасности» согласует [стороннюю библиотеку](#) с точки зрения безопасности.

Правила

1. Срок выполнения — 3 рабочих дня.
2. В каждом конкретном случае решение принимается на усмотрение тестировщика.

Порядок действий

Ознакомьтесь с предлагаемой библиотекой и оцените возможность ее использования с точки зрения безопасности. В дополнительных полях указаны описание и ссылка на новость, в которой обсуждалась целесообразность ее использования.

Алгоритм оценки безопасности

По умолчанию считается, что библиотека не проходит проверку безопасности, если инструмент (для каждого свой) показывает две и больше критических уязвимостей.

В каждом конкретном случае принимайте решение по своему усмотрению. Поскольку:

- В вашем случае уязвимости могут быть незначительными или неприменимыми.
- Вы можете разрешить конкретному разработчику использовать библиотеку «под свою ответственность», так как в библиотеке могут быть отдельные опасные функции, без применения которых использование библиотеки будет безопасным.
- Возможна обратная ситуация — уязвимость средней критичности может напрямую затрагивать безопасность продукта.

Учтите, что инструменты оценки могут давать не полную картину уязвимостей, поскольку базы не обновляются сразу с появлением новой уязвимости. Поэтому, при тщательной проверке, стоит использовать дополнительные источники.

Node.js

У node.js существует проверка зависимостей во встроенном менеджере пакетов npm.

Обычно на проверку приходят задачи с полным именем и версией пакета (например, `@reduxjs/toolkit@1.8.0`), поэтому можно ввести: `npm install @reduxjs/toolkit@1.8.0`

Эта команда установит пакет и автоматически проверит его на уязвимости. Таким же образом можно вводить названия нескольких пакетов: `npm install @reduxjs/toolkit@1.8.0 @types/react-redux@7.1.23`

Python

Для python есть несколько инструментов проверки зависимостей, но я остановился на стандартном инструменте `pip-audit` и удобном `safety`.

`Safety` более удобен форматом ввода — можно вводить названия зависимостей, не создавая отдельного файла, что удобно при проверке небольшого количества библиотек: `echo "insecure-package==0.1" | safety check --stdin`

При этом [pip-audit](#) полностью бесплатен, но документация у него хуже и нет возможности «быстрой» проверки одного пакета, как в примере с [safety](#).

JS

У проекта [Retire.js](#) есть множество способов поиска уязвимых JS библиотек. Этот инструмент стоит использовать для поиска уязвимостей в библиотеках, которые добавляются для нашего интерфейсного фреймворка. [Репозиторий Retire.js](#).

Maven

Maven используется для хранения библиотек, которые используются в мобильных приложениях на android. Для проверки используется OWASP Dependency-Check — плагин, который подключается прямо в pom-файле проекта.

Пример файла pom.xml для проверки пакета org.springframework:spring-core:5.0.1.RELEASE:

```
1 <project>
2   <modelVersion>4.0.0</modelVersion>
3   <groupId>com.mycompany.app</groupId>
4   <artifactId>my-app</artifactId><version>1</version>
5   <build>
6     <plugins>
7       <plugin>
8         <groupId>org.owasp</groupId>
9         <artifactId>dependency-check-maven</artifactId>
10        <version>7.2.1</version>
11        <executions>
12          <execution>
13            <goals>
14              <goal>check</goal>
15            </goals>
16          </execution>
17        </executions>
18      </plugin>
19    </plugins>
20  </build>
21  <dependencies>
22    <dependency>
23      <groupId>org.springframework</groupId>
24      <artifactId>spring-core</artifactId>
25      <version>5.0.1.RELEASE</version>
26    </dependency>
27  </dependencies>
28 </project>
```

Подробнее о проверке проектов через Dependency-Check:

<https://github.com/jeremylong/DependencyCheck>

<https://itnext.io/owasp-dependency-check-maven-vulnerabilities-java-898a9cf99f5e>

<https://sorenpoulsen.com/using-owasp-dependency-check-with-maven> (доступен через VPN)

C++

На данный момент не найдены полностью бесплатные и подходящие инструменты для проверки библиотек для C++. Но для наших целей может подойти snyk — он позволяет сканировать C++ библиотеки на наличие уязвимостей, сравнивая файлы в директориях с файлами потенциально уязвимых библиотек из своей базы.

Пример запуска, после которого будут протестированы все файлы в текущей директории: `snyk test --unmanaged`

[Подробнее.](#)

Закройте согласование

- Если вы разрешаете использование библиотеки, нажмите «Согласовано».
- Если вы не согласны с использованием данной библиотеки, в комментарии укажите причину отказа в согласовании и нажмите «Отказано».

Итог

Принято решение по использованию сторонней библиотеки.

